

Ejercicios Repaso Tema 9

Bloque 1: Dominio de ArrayList y Operaciones Agregadas (RA6 c, d, j)

Ejercicio 1: Estadísticas de una Lista

Escribe un programa que introduzca una cantidad aleatoria (entre 10 y 20 elementos) de valores numéricos enteros aleatorios (entre 0 y 100) en un `ArrayList<Integer>`. A continuación, utilizando un bucle estilo *for-each* (iterador), calcula y muestra por pantalla la suma total, la media aritmética, el valor máximo y el valor mínimo de esos números.

Ejercicio 2: Filtrado Dinámico

Crea un programa que gestione una lista de palabras utilizando un `ArrayList<String>`. Añade 8 palabras diferentes de forma manual. Utiliza el método `removeIf()` para eliminar de la lista todas aquellas palabras que contengan la letra "a". Muestra la lista antes y después del borrado.

Ejercicio 3: Ordenación de Objetos (Comparable)

Basándote en el ejemplo de gestión de objetos, crea una clase `Gato` con los atributos `nombre`, `color` y `raza`. Haz que la clase implemente la interfaz `Comparable<Gato>` para que su orden natural sea alfabéticamente por el atributo `nombre`. En el programa principal, crea un `ArrayList` con 5 gatos, ordénalos utilizando `Collections.sort()` y muéstralos por pantalla iterando la colección.

Bloque 2: Dominio de HashMap (RA6 e, j)

Ejercicio 4: Diccionario Bilingüe y Control de Acceso

Crea un mini-diccionario español-inglés utilizando un `HashMap<String, String>` que contenga al menos 10 palabras predefinidas. El programa debe pedir al usuario una palabra en español y devolver su traducción al inglés recuperando el valor mediante la clave. Si la palabra no existe (es decir, el diccionario devuelve `null` o `containsKey` es falso), debe mostrar el mensaje "No conozco esa palabra".

Ejercicio 5: Control de Acceso (Login)

Implementa un sistema de control de acceso a un área restringida. Los nombres de usuario y sus correspondientes contraseñas deben estar almacenados como pares clave-valor en un `HashMap<String, String>`. El programa pedirá usuario y contraseña, permitiendo un máximo de 3 intentos. Si acierta, mostrará "Ha accedido al área restringida"; si agota los intentos, mostrará "Lo siento, no tiene acceso al área restringida".

Ejercicio 6: Calculadora de Puntuación (Ajedrez)

En ajedrez, las piezas tienen un valor: dama (9), torre (5), alfil (3), caballo (2) y peón (1). Crea un `HashMap<String, Integer>` para almacenar esta correspondencia. Luego, genera al azar una secuencia de entre 0 y 15 piezas capturadas por un jugador. Muestra la lista de piezas capturadas (iterando sobre ella) y calcula la puntuación total sumando los valores recuperados del diccionario.

Bloque 3: Integración de ArrayList y HashMap (RA6 c, d, e, j)

Ejercicio 7: El Carrito de la Compra

Un supermercado almacena su catálogo de productos y precios en un `HashMap<String, Double>` (por ejemplo: "tomate" -> 1.59, "quinoa" -> 4.50). Crea una clase `Elemento` que represente una línea de compra (nombre del producto y cantidad). El carrito de la compra será un `ArrayList<Elemento>`. El programa debe permitir al usuario añadir elementos al carrito hasta que escriba la palabra "fin". Al terminar, debe recorrer la lista del carrito, cruzar la información con el diccionario para obtener el precio, calcular el subtotal de cada línea y el coste total de la compra.

Ejercicio 8: Registro de Movimientos Bancarios

Crea un sistema bancario donde un `HashMap<String, ArrayList<Double>>` asocie un número de cuenta corriente (Clave) con una lista de todos sus movimientos económicos (Valor).

1. Registra 3 cuentas con movimientos predefinidos (ingresos positivos, cargos negativos).
2. Pide al usuario un número de cuenta.
3. Si existe, itera sobre su `ArrayList` de movimientos para imprimir el historial y calcular el saldo actual (operación agregada).

Bloque 4: Clases y Métodos Genéricos (RA6 f)

Ejercicio 9: La Clase Genérica `Registro<T>`

Crea una clase genérica llamada `Registro<T>` que simule una entrada en una base de datos. Debe tener dos atributos: un `id` de tipo `int` y un `dato` de tipo genérico `T`.

1. Implementa su constructor y el método `toString()`.
2. En el programa principal, crea un `ArrayList` que almacene objetos `Registro<String>` (para guardar nombres de clientes) y otro `ArrayList` que almacene `Registro<Gato>` (reutilizando la clase del Ejercicio 3). Muestra el contenido de ambas listas.

Ejercicio 10: Método Genérico de Extracción

Crea una clase `UtilidadesColecciones` con un método estático genérico llamado `obtenerUltimoElemento`. Este método debe recibir por parámetro un `ArrayList<E>` de cualquier tipo y devolver el último elemento de la lista (utilizando el método `.size() - 1`). Si la lista está vacía (`isEmpty()`), debe devolver `null`. Ponlo a prueba pasándole tanto una lista de `Integer` como una lista de `String`.